



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Sistema de Almoxarifado

Gestão de Estoque e Organização de ativos

Integrantes:

Guilherme Caio Pimentel

José Henrique Pereira Brito

Matheus Trindade Rocha

Pedro Henrique Correa Diniz

Rodrigo Gonçalves Giancott Magalhães

Disciplina: Programação
Orientada por Objeto

O problema

O desafio do Controle de Estoque

Efeitos da falta de um sistema de gestão de estoque:

- Controle manual sujeito a erros
- Dificuldade no acompanhamento de entradas e saídas
- Divergência entre estoque físico e registros
- Perda ou falta de materiais importantes
- Baixa eficiência no gerenciamento de fornecedores

A solução

Como organizar um espaço e seus conteúdos



Diante a necessidade de organização de um espaço para o controle de seus insumos, tomamos por criar um sistema responsável por cadastrar, armazenar, visualizar e alterar a quantidade e informações de diferentes insumos.

Para isso, o sistema utilizará da linguagem c# para montagem do sistema, além de tecnologias como JSON server para armazenar dados e arquivos de texto (.txt) para impressão de relatórios de entrada e saída para o usuário.

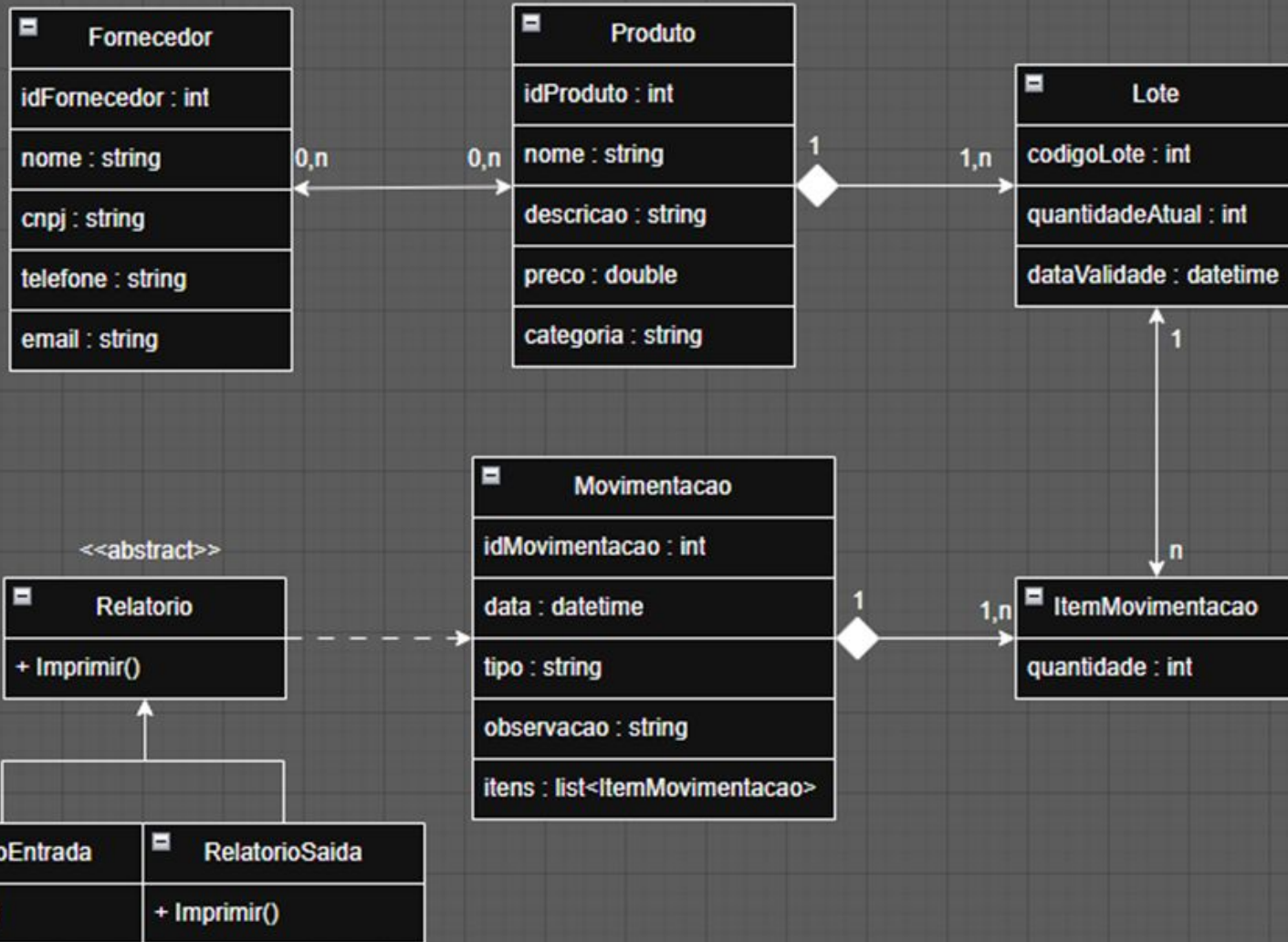


Diagrama UML mostrando o relacionamento entre as classes presentes no sistema

Pilares da POO: Encapsulamento

“Encapsular significa ‘proteger’ os dados dentro de um objeto, evitando acessos diretos e não autorizados. Isso é feito controlando o que pode ou não ser visto ou alterado por outras partes do sistema.”

```
1 reference
private int idProduto;
2 references
private string nome;
2 references
private string descricao;
2 references
private double preco;
2 references
private string categoria;
2 references
private List<Fornecedor> fornecedores;
2 references
private List<Lote> lotes;
```

```
0 references
public void AdicionarFornecedor(Fornecedor fornecedor) ...

0 references
public void AdicionarLote(Lote lote) ...

1 reference
public string ObterNome() ...

0 references
public string ObterDescricao() ...

0 references
public double ObterPreco() ...

public string ObterCategoria() ...
```

Encapsulamento e segurança dos valores dos atributos dentro da classe Produto

Pilares da POO: Abstração

“A abstração consiste em um dos pontos mais importantes dentro de qualquer linguagem Orientada a Objetos. Como estamos lidando com uma representação de um objeto real (o que dá nome ao paradigma), temos que imaginar o que esse objeto irá realizar dentro de nosso sistema. São três pontos que devem ser levados em consideração nessa abstração: identidade, propriedade e métodos.”

```
4 references
public abstract class Relatorio
{
    2 references
    protected void Cabecalho(StreamWriter writer)
    {
        writer.WriteLine("=== RELATÓRIO ALMOXARIFADO ===");
        writer.WriteLine($"DATA DE EMISSÃO: {DateTime.Now}");
    }

    2 references
    public abstract void Imprimir();
}

1 reference
public class RelatorioEntrada : Relatorio
{
    6 references
    private Movimentacao movimentacao;

    0 references
    public RelatorioEntrada(Movimentacao movimentacao)
    {
        this.movimentacao = movimentacao;
    }

    1 reference
    public override void Imprimir() ...
}
```

*Abstração da
classe Relatorio
para definir o
comportamento
da classe-filha*

Pilares da POO: Herança

“O reuso de código é uma das grandes vantagens da programação orientada a objetos. Muito disso se dá por uma questão que é conhecida como herança. Essa característica otimiza a produção da aplicação em tempo e linhas de código.”

RelatorioEntrada herdando de Relatorio o comportamento Imprimir() e o método Cabecalho()

```
1 reference
public class RelatorioEntrada : Relatorio
{
    6 references
    private Movimentacao movimentacao;

    0 references
    public RelatorioEntrada(Movimentacao movimentacao)
    {
        this.movimentacao = movimentacao;
    }

    1 reference
    public override void Imprimir()
    {
        using(StreamWriter writer = new StreamWriter("RelatorioEntrada.txt"))
        {
            Cabecalho(writer);

            writer.WriteLine("RELATÓRIO DE ENTRADA");
            writer.WriteLine();
        }
    }
}
```


Pilares da POO: Polimorfismo

“Polimorfismo significa que objetos diferentes podem responder de formas distintas ao mesmo comando. Isso permite criar sistemas flexíveis, onde cada objeto sabe como se comportar sem que o programador precise escrever blocos de código específicos para cada um. É como dizer “mover” para um animal, e o cachorro correr, o peixe nadar e o pássaro voar.”

```
Relatorio entrada = new RelatorioEntrada();  
Relatorio saida = new RelatorioSaida();
```

```
entrada.Imprimir();  
saida.Imprimir();
```



RelatorioEntrada.txt



RelatorioSaida.txt

Polimorfismo dinâmico (sobrescrita) na classe Relatorio

Demonstração do Código em Execução

Fontes:

O problema:

<https://zeusautomacao.com.br/5-problemas-gerados-pela-falta-de-gerenciamento-do-estoque/>

<https://almoxarifenapratica.com.br/problemas-no-almoxarifado-veja-os-7-problemas-mais-comuns/>

Pilares da POO:

Material didático providenciado através do Canvas

<https://www.fiap.com.br/radar-tech/post/programacao-orientada-a-objetos-poo-um-guia-completo-sobre-seus-4-pilares-fundamentais-encapsulamento-heranca-polimorfismo-e-abstracao/>

<https://www.devmedia.com.br/os-4-pilares-da-programacao-orientada-a-objetos/9264>

Obrigado por assistir!

